Quiz 06 - Polymorphism

Name _____ Date _____

Multiple choice. Choose the **one** best answer for each question and circle it on the **answer sheet** (page 5). All code is from Worksheet 06. Every snippet compiles unless a question says otherwise. About 10 minutes.

part 1 Static vs. Dynamic Binding

```
class Vehicle {
public:
    void describe() { cout << "A vehicle.\n"; }
};
class Car : public Vehicle {
public:
    void describe() { cout << "A car.\n"; }
};

int main() {
    Car c;
    Vehicle* vp = &c;      // base-class POINTER to a Car
    Vehicle& vr = c;       // base-class REFERENCE to a Car

    c.describe();          // (a)
    vp->describe();        // (b)
    vr.describe();         // (c)
}
```

1. As written, what does line (b) `vp->describe();` print?

   Ⓐ `A car.`
   Ⓑ `A vehicle.`
   Ⓒ `A vehicle.` then `A car.`
   Ⓓ Nothing. It's a compile error.

2. `Vehicle::describe()` is changed to `virtual void describe()`. Now what does line (c) `vr.describe();` print?

   Ⓐ `A car.`
   Ⓑ `A vehicle.`
   Ⓒ `A vehicle.` then `A car.`
   Ⓓ Nothing. It's a compile error.

3. Adding `virtual` changes exactly two of the three lines. Which call does *not* change?

   Ⓐ (b) only
   Ⓑ (c) only
   Ⓒ None. All three change.
   Ⓓ (a) only

4. Without `virtual`, the compiler picks which `describe()` to run from…

   Ⓐ the actual object's type, at run time
   Ⓑ the order the classes were declared
   Ⓒ the pointer's or reference's declared type, at compile time
   Ⓓ the most recently constructed object

5. Which call has the version that runs chosen at **run time**?

   Ⓐ `for (auto& s : shapes) s->draw();` where `draw()` is `virtual`
   Ⓑ `print(5);` where `print(int)` and `print(string)` both exist
   Ⓒ `Fraction c = a + b;` using the `operator+` you wrote in W04
   Ⓓ `Vehicle* v = new Car(); v->describe();` where `describe()` is **not** virtual

Quiz 06 - Polymorphism

part 2 `virtual`, `override`, and the Signature Rules

Treat each numbered line as the *only* function in `Wizard`.

```
class Character {
public:
    virtual ~Character() {}
    virtual void attack();
    virtual void printName() const;
    virtual void heal(int amount);
    void rest();                          // NOT virtual
};

class Wizard : public Character {
public:
    void attack() override;               // (1)
    void printName() override;            // (2)
    void heal(double amount) override;    // (3)
    void rest() override;                 // (4)
    void Attack() override;               // (5)
    void heal(int amount);                // (6)  no 'override' keyword
};
```

6. Why is line (2) a compile error?

Ⓐ `printName()` isn't virtual in `Character`

Ⓑ The name doesn't match

Ⓒ The parameter types don't match

Ⓓ It's missing `const`

7. Why is line (4) a compile error?

Ⓐ It compiles and overrides fine

Ⓑ `rest()` isn't virtual in `Character`

Ⓒ It's missing `const`

Ⓓ The parameter types don't match

8. You delete the word `override` from line (2), so it now compiles. Given `Character* p = new Wizard();`, which version does `p→printName();` run?

Ⓐ `Character::printName()`. Wizard's version hides it instead of overriding it.

Ⓑ `Wizard::printName()`

Ⓒ Both, base version first

Ⓓ It crashes at run time

Quiz 06 - Polymorphism

part 3 Pointers, References, and Slicing

```
class Character {
protected:
    string name;
public:
    Character(string n) : name(n) {}
    virtual ~Character() {}
    virtual void printName() { cout << name << endl; }
};
class Wizard : public Character {
public:
    Wizard(string n) : Character(n) {}
    void printName() override { cout << "The magnificent " << name << endl; }
};
class Warrior : public Character {
public:
    Warrior(string n) : Character(n) {}
    void printName() override { cout << "The powerful " << name << endl; }
};

void byRef(Character& c) { c.printName(); }
void byPtr(Character* c) { c->printName(); }
void byVal(Character c)  { c.printName(); }

int main() {
    Wizard  merlin("Merlin");
    Warrior conan("Conan");

    byRef(merlin);                                  // (a)
    byPtr(&conan);                                  // (b)
    byVal(merlin);                                  // (c)

    vector<Character>  copies = {merlin, conan};
    copies[1].printName();                          // (d)

    vector<Character*> party  = {&merlin, &conan};
    party[1]->printName();                          // (e)
}
```

9. What does line (c) print?

- (A) `The magnificent Merlin`
- (B) `The powerful Merlin`
- (C) Nothing. It's a compile error.
- (D) `Merlin`

10. What does line (d) print?

- (A) `The powerful Conan`
- (B) `Conan`
- (C) `Merlin`
- (D) `The magnificent Merlin`

11. What does line (e) print?

- (A) `Conan`
- (B) `The magnificent Merlin`
- (C) `The powerful Conan`
- (D) Nothing. It's a compile error.

12. Lines (c) and (d) both print the plain name even though `printName()` is virtual. What is the one-character change to `byVal` that fixes (c)?

- (A) Change the parameter to `const Character c`
- (B) Change the parameter to `Character& c`
- (C) Add `override` to `Character::printName()`
- (D) Change `c.printName()` to `c->printName()`

Quiz 06 - Polymorphism

part 4 Virtual Destructors

```
class Base {
public:
    ~Base() { cout << "~Base "; }                  // not virtual
};

class Derived : public Base {
    int* data;
public:
    Derived() : data(new int[100]) {}
    ~Derived() { cout << "~Derived "; delete[] data; }
};

int main() {
    Base* p = new Derived();
    delete p;
}
```

13. As written, what does the program print?

- (A) `~Derived ~Base`
- (B) `~Base`
- (C) `~Base ~Derived`
- (D) Nothing

14. The fix is one word. Which word, and where does it go?

- (A) `virtual` in front of `~Derived()`
- (B) `delete[]` in place of `delete` in `main()`
- (C) `virtual` in front of `~Base()`
- (D) `override` after `~Derived()`

15. After the fix, what does the program print?

- (A) `~Base ~Derived`
- (B) `~Base`
- (C) `~Derived`
- (D) `~Derived ~Base`

16. Finish the rule of thumb: “If a class has ____, give it a ____.”

- (A) any virtual functions; virtual destructor
- (B) a constructor; destructor
- (C) pointer members; const method
- (D) a base class; copy constructor

Quiz 06 - Polymorphism

Name _____ Date _____

answers **Answer Sheet**

Mark (color in) one letter per question

1.	A	B	C	D	9.	A	B	C	D
2.	A	B	C	D	10.	A	B	C	D
3.	A	B	C	D	11.	A	B	C	D
4.	A	B	C	D	12.	A	B	C	D
5.	A	B	C	D	13.	A	B	C	D
6.	A	B	C	D	14.	A	B	C	D
7.	A	B	C	D	15.	A	B	C	D
8.	A	B	C	D	16.	A	B	C	D